UI REQUIREMENTS SPECIFICATION

Identity & Access Management

iam-staff-ui

Consumes REST APIs exposed by iam-staff-portal-api

Prepared for engineering · July 16, 2026

OVERVIEW

Purpose & Scope

1

Single-page admin console

iam-staff-ui is the internal console operations staff use to manage every application registered in the IAM platform, and the login providers that authenticate their users.

2

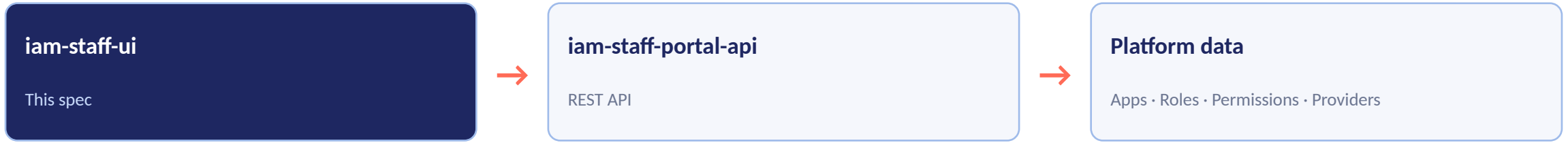
API-driven

Every screen reads and writes through the REST API surface of iam-staff-portal-api — no direct database access from the UI.

3

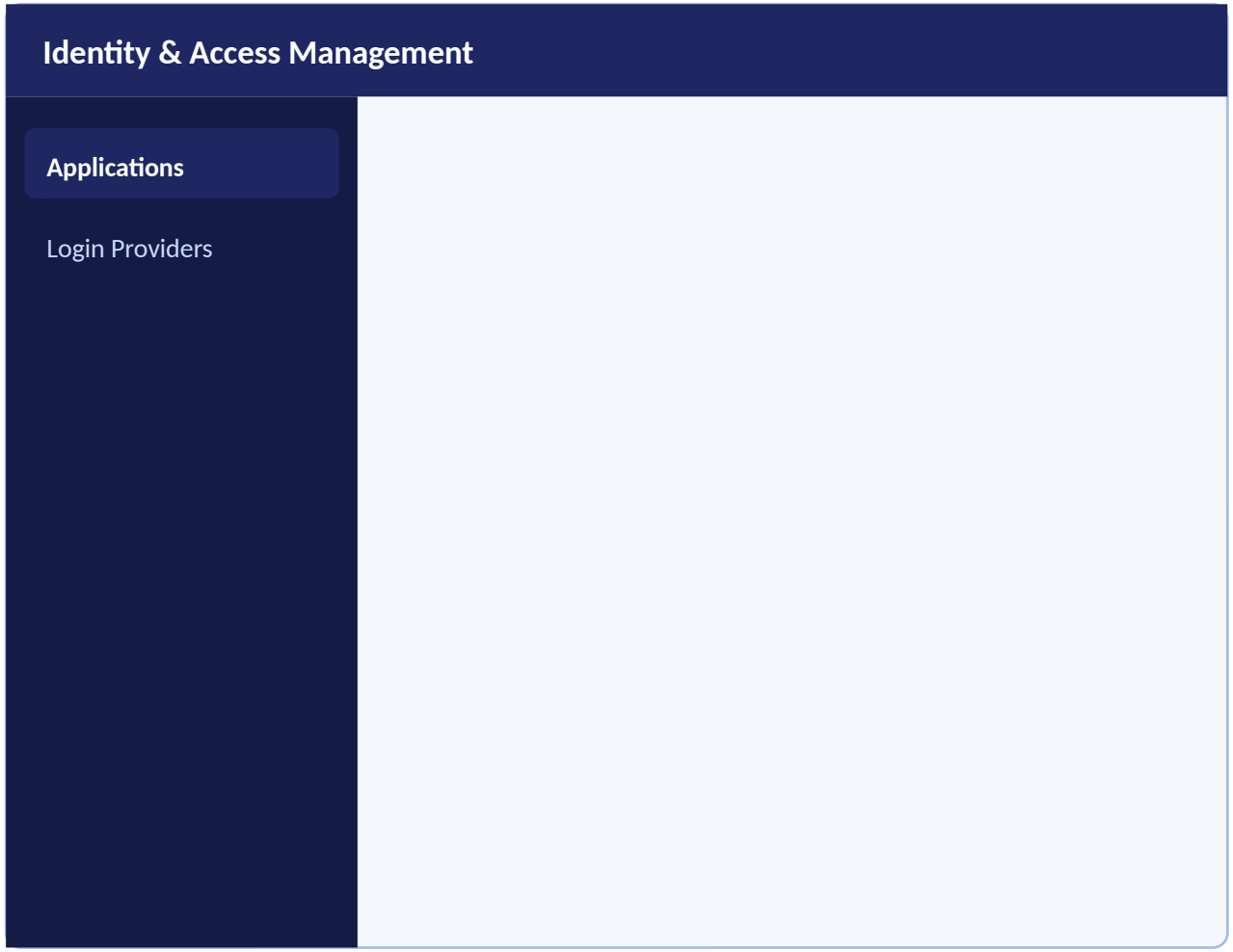
Two top-level areas

A left-hand navigation exposes exactly two sections: Applications and Login Providers, described on the following slides.



GLOBAL SHELL

Application Title & Left Navigation



Wireframe — left nav (illustrative)

App title

Every screen renders the fixed application title “Identity & Access Management” in the top bar.

Left menu — 2 items

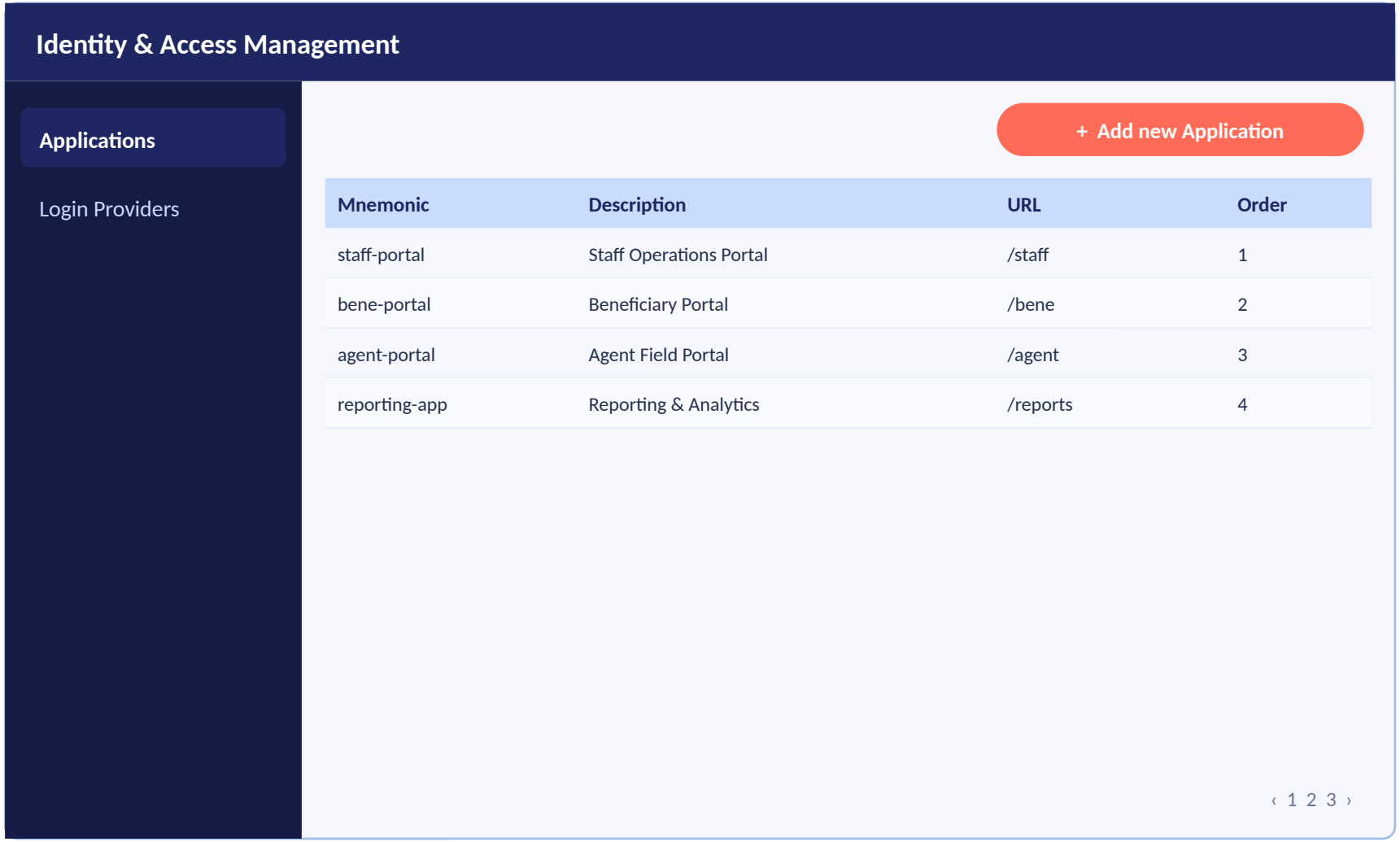
Applications and Login Providers are the only entries in the left-hand menu, always visible, indicating the active section.

Persistent chrome

Title bar and menu remain fixed while the content area to the right changes per section.

APPLICATIONS · LIST VIEW

Applications — List View



Model: StaffPortalApplication

Backed by the application registry — one row per registered staff-side application.

Pagination

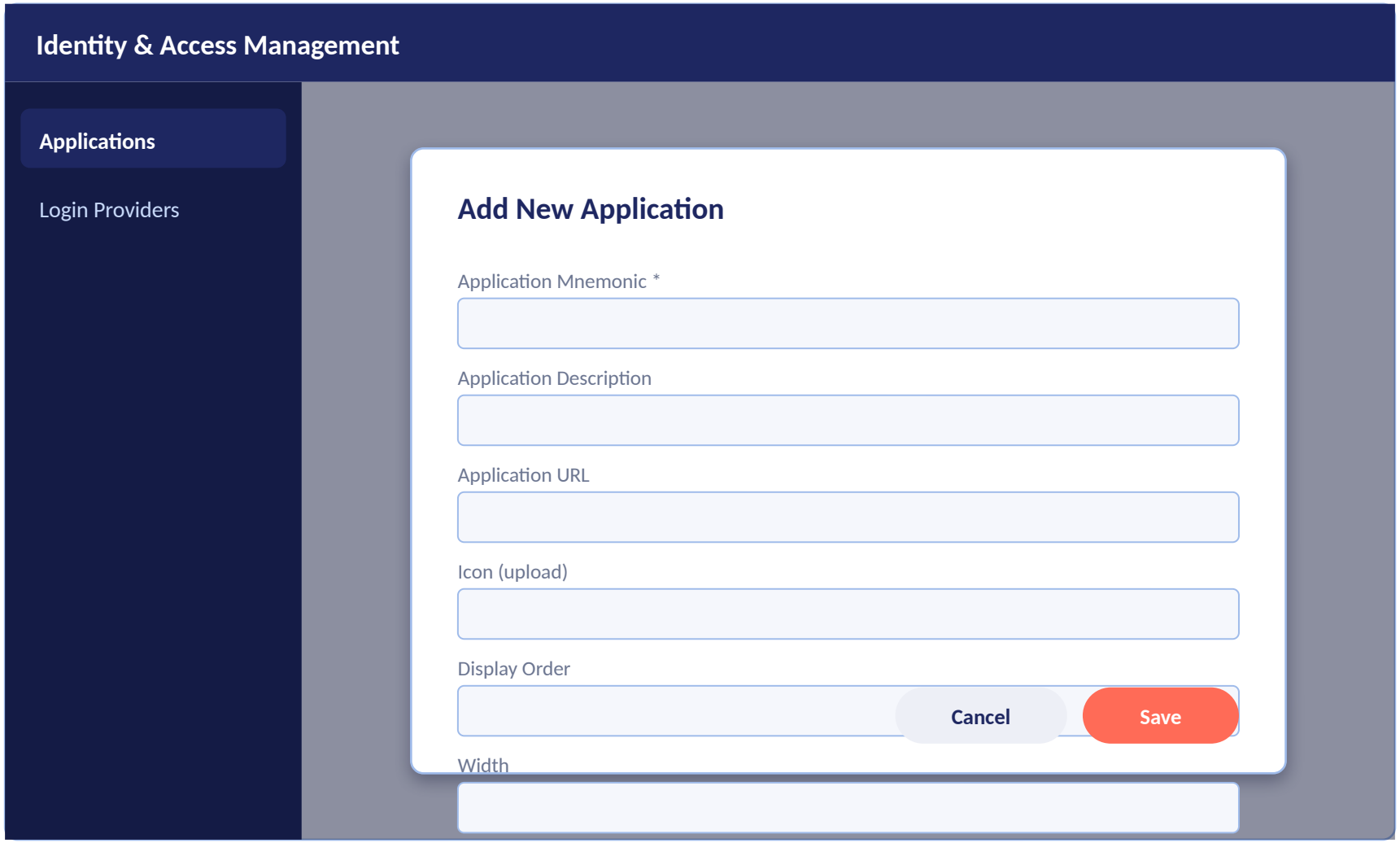
Standard page controls anchored to the bottom of the list.

Row click

Selecting a row opens the Application Detail View (next slide).

APPLICATIONS · CREATE

Add New Application — Popup



Fields = StaffPortalApplication

application_mnemonic (unique, required), application_description, application_url, icon_base64, order, width.

is_self_registered

System-managed flag; not editable from this form — set by the API when an app self-registers.

On Save

Modal closes → user returns to the Applications list → new record appears at the top of the list.

APPLICATIONS · DETAIL VIEW

Application Detail — 5 Tabs

Clicking a row in the Applications list opens the Detail View for that application, organized into five tabs:

1 Application

All attributes of the selected application (read/edit view of StaffPortalApplication).

2 Roles

Roles defined for this application — list, add, delete.

3 Permissions

Permissions defined for this application — list, add, delete.

4 Roles ↔ Permissions

Mappings between roles and permissions, filterable by either side.

5 Data Policies

Data-scoping policies for this application — list, add, remove.

DETAIL VIEW · TAB 1 OF 5

Application Tab — Attributes

Application | Roles | Permissions | Roles ↔ Perms | Data Policies

`application_mnemonic`	String, unique, required — stable key identifying the application.
`application_description`	Free-text description shown throughout the UI.
`application_url`	Base URL the staff portal links to / launches.
`icon_base64`	Base64-encoded icon rendered next to the application name.
`width`	Optional integer — layout hint for embedding the app tile.
`order`	Optional integer controlling display order in lists.
`is_self_registered`	Boolean, system-set — true if the app registered itself via API rather than being seeded.

This tab renders every field of the StaffPortalApplication model shown above; secret/binary fields are not applicable to this model.

DETAIL VIEW · TAB 2 OF 5

Roles Tab — List, Add, Delete

Application | **Roles** | Permissions | Roles ↔ Perms | Data Policies

+ Add Role

Role Mnemonic	Description		
APP_ADMIN	Full administrative access	–	× delete
APP_VIEWER	Read-only access	–	
APP_APPROVER	Can approve workflow items	–	

‹ 1 2 3 ›

Model: StaffRole

role_mnemonic, role_description, application_id (implicit — this application).

Add

“Add Role” opens a popup to create a role scoped to this application.

Delete

Each row has an inline delete action to remove that role.

Pagination

Standard bottom pagination for the roles list.

DETAIL VIEW · TAB 3 OF 5

Permissions Tab — List, Add, Delete

Application | Roles | **Permissions** | Roles ↔ Perms | Data Policies

+ Add Permission

Permission Mnemonic	Description		
users.read	View user records	–	× delete
users.write	Create / update user records	–	
reports.export	Export reporting data	–	

‹ 1 2 3 ›

Model: StaffApplicationPermission

permission_mnemonic, permission_description, application_id (implicit — this application).

Add

“Add Permission” opens a popup to create a permission scoped to this application.

Delete

Each row has an inline delete action to remove that permission.

Pagination

Standard bottom pagination for the permissions list.

DETAIL VIEW · TAB 4 OF 5

Roles ↔ Permissions Tab — Mapping & Filters

Application | Roles | Permissions | **Roles ↔ Perms** | Data Policies

Filter: Role ▾ Permission ▾ + Map Role → Permission

Role	Permission		
APP_ADMIN	users.read	–	× remove
APP_ADMIN	users.write	–	
APP_VIEWER	users.read	–	

‹ 1 2 3 ›

Model: StaffRolePermission

role_id, permission_id — join table between StaffRole and StaffApplicationPermission.

Dual filter

Filter by Role to see its permissions, or by Permission to see which roles grant it.

Add / remove

Create a new mapping via popup; remove an existing mapping inline.

Pagination

Standard bottom pagination for the mapping list.

DETAIL VIEW · TAB 5 OF 5

Data Policies Tab — List, Add, Remove

Application | Roles | Permissions | Roles ↔ Perms | **Data Policies**

+ Add Data Policy

Data Policy Mnemonic	Description	
REGION_NORTH	Restrict data to Northern region	– × remove
OWN_RECORDS_ONLY	Restrict to records owned by the user	–

‹ 1 2 3 ›

Implementation note: data policies are not a separate table today — they are StaffRole rows whose role_mnemonic is prefixed “DP_” (see DataPolicyMiddleware / DP_ROLE_PREFIX in iam-core). The UI should present them as a distinct tab, stripping the DP_ prefix for display.

Source

Backed today by StaffRole records carrying the DP_ prefix, scoped to this application.

Add

“Add Data Policy” opens a popup; UI applies the DP_ prefix behind the scenes.

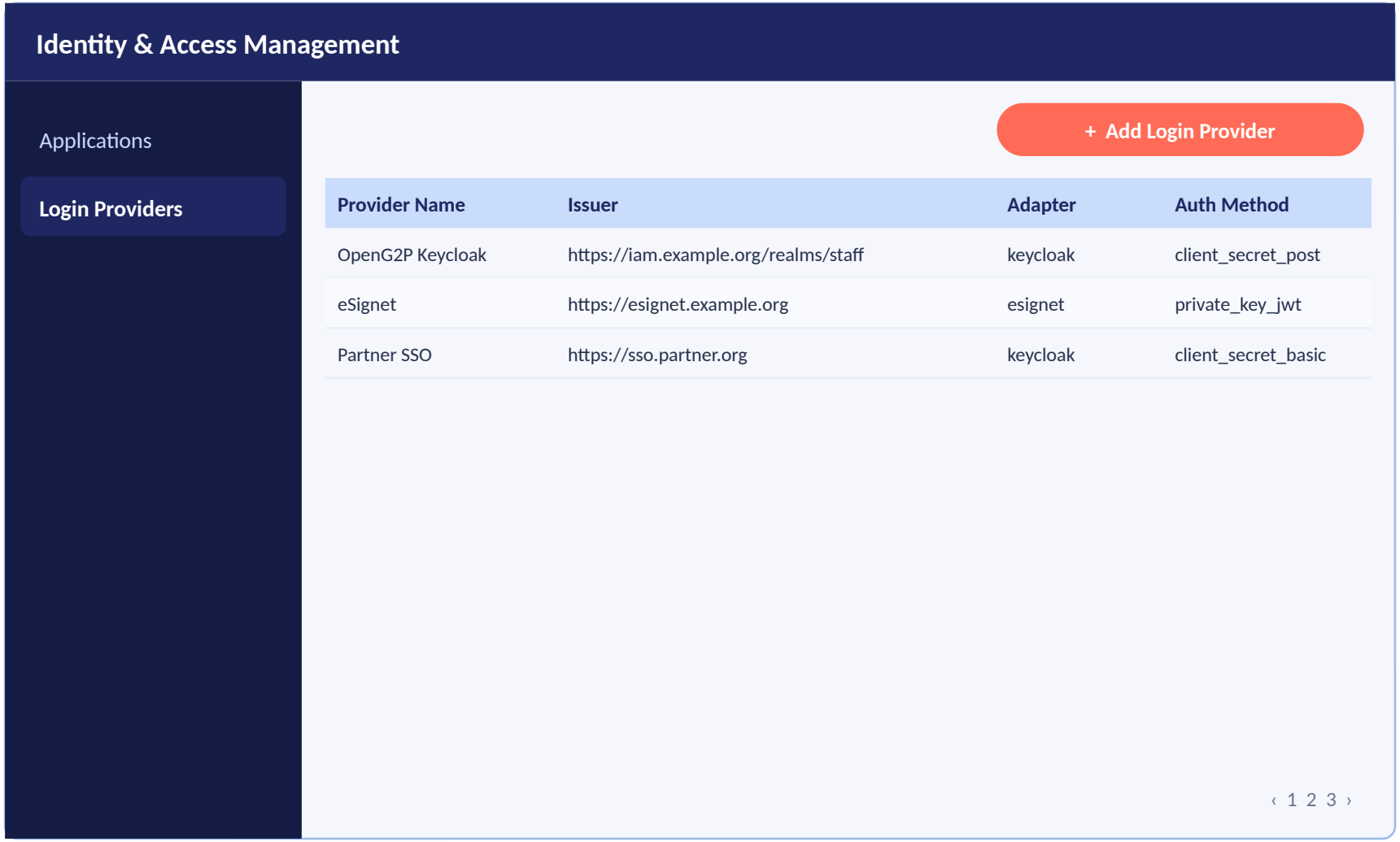
Remove

Inline remove action per row.

Pagination

Standard bottom pagination.

Login Providers — List View



Model: LoginProvider

One row per OIDC/OAuth identity provider staff can authenticate against.

Pagination

Standard bottom pagination.

Row click

Opens the provider for view/edit (same field set as the create popup).

Add New Login Provider — Popup Field Groups

The create/edit popup exposes every field of the LoginProvider model, grouped for usability:

Identity

- `provider_name`
- `description`
- `icon_base64`

OAuth Client

- `client_id`
- `client_secret`
- `client_private_key`
- `token_endpoint_auth_method`

OIDC Endpoints

- `issuer`
- `authorization_endpoint`
- `token_endpoint`
- `userinfo_endpoint`
- `server_metadata_url`
- `jwks_uri`

Behavior & Routing

- `adapter_name`
- `scope`
- `enable_pkce`
- `extra_authorize_params`
- `jwt_assertion_aud`
- `audiences`
- `oauth_callback_url`
- `default_redirect_uri`

Also captured (system-managed, read-only in UI): keymanager_app_id, keymanager_ref_id.

APPENDIX

Screen → Backend Model Reference

Screen	Backend model	Source
Applications list & Application tab	StaffPortalApplication	iam_staff_portal_api/models
Roles tab	StaffRole	iam_staff_portal_api/models
Permissions tab	StaffApplicationPermission	iam_staff_portal_api/models
Roles ↔ Permissions tab	StaffRolePermission	iam_staff_portal_api/models
Data Policies tab	StaffRole (role_mnemonic prefixed “DP_”)	iam_core / data_policy_role_helper
Login Providers list & popup	LoginProvider	iam_core/models

All models extend BaseORMModelWithTimes, so every record also carries the platform-standard system fields (e.g. id, created/updated timestamps). These are not shown as editable UI fields except where noted.

WRAP-UP

Assumptions & Open Questions

Assumptions

- New Application is inserted at the top of the list — assumes API returns/allows client-side sort by most-recently-created, or the UI re-fetches sorted by created_at desc.
- Delete/remove actions on Roles, Permissions, mappings, and Data Policies are assumed irreversible from the UI and should carry a confirmation step.
- Data Policies are modeled as DP_-prefixed StaffRole rows today; if a dedicated model is introduced later, this tab's data source should be swapped without changing its UX.

Open Questions

- iam-staff-portal-api currently exposes auth-only endpoints (auth, identity provider, OAuth callback, user access); CRUD endpoints for Applications, Roles, Permissions, and mappings will need to be added to support this UI.
- client_secret and client_private_key on Login Providers are sensitive — UI should mask/never redisplay secrets after save.
- Pagination page size and default sort order to be confirmed with backend team per list.